Available online at www.sciencedirect.com
SCIENCE DIRECT®
ELSEVIER
Discrete Applied Mathematics 133 (2004) 69–92
DISCRETE APPLIED MATHEMATICS
www.elsevier.com/locate/dam

A multi-phase constructive heuristic for the vehicle routing problem with multiple trips☆

R.J. Petch, S. Salhi

Management Mathematics Group, School of Mathematics and Statistics, The University of Birmingham, UK

Received 10 October 2000; received in revised form 20 September 2002; accepted 15 December 2002

Abstract

An extension to the classical vehicle routing problem where vehicles can be assigned more than one route within a working time period is investigated. A multi-phase constructive heuristic, which is enhanced by suitable data structure, is proposed. Results are given comparing to benchmarks from the literature.
© 2003 Elsevier B.V. All rights reserved.

Keywords: Heuristic; Population; Bin-packing; Multiple trip

1. Introduction

The problem within distribution management of scheduling vehicles from one or more fixed positions (depots) to service a given set of locations (customers) is called the vehicle routing problem (VRP). The scheduling process involves designing routes for the vehicles, subject to given constraints, in order to achieve the objective of minimizing the total cost of delivery. This paper addresses a VRP where the classical assumption of a one to one correspondence between routes and vehicles is removed. Known as the vehicle routing problem with multiple trips (VRPM), the problem is characterized by vehicles and hence drivers working multiple routes or trips within a given time period. In practice multiple trip scheduling is important since significant cost savings can be achieved if the number of vehicles and hence drivers is reduced. In some cases, a more suitable solution to the distribution problem may be to consider a larger

☆ An earlier version of this paper was presented at *CO*2000, London and *EUROXVII*, Budapest.

E-mail address: salhis@for.mat.bham.ac.uk (S. Salhi).

0166-218X/$ - see front matter © 2003 Elsevier B.V. All rights reserved.
doi:10.1016/S0166-218X(03)00434-7

set of vehicles which would reduce the problem to a classical-type VRP. Nevertheless, this is not always possible due to setup costs or constraints such as road access. The VRPM can be used for both strategic and tactical planning. Because a reduced vehicle fleet size might be more desirable, a strategic VRPM objective accounts for both vehicle and scheduling costs. In this situation, there might be a trade off between higher scheduling costs and lower vehicle and driver associated costs, the latter being usually more significant. In practice, within a tactical distribution system, the fleet might be fixed to the existing setup and so the objective is reduced to minimizing scheduling costs.

This paper describes a proposed multi-phase constructive heuristic for the solution of the VRPM. In order to compare against benchmarks achieved from the literature a similar objective is chosen, i.e. the minimization of the maximum overtime restriction for a prescribed minimum vehicle fleet size. The next section provides a brief review of the literature. This is followed by a presentation of the main aspects of the algorithm, its motivation as well as notation and objective before the key features of the methodology are described in detail. Finally, computational results are given comparing against published results from the literature.

2. Literature review

Although in practice multiple route assignment is common, there is a shortage of papers covering this feature. The first work to explicitly address multiple trips was made by Salhi [11] in the context of vehicle fleet mix. Limited to double trips, a matching algorithm is used to allocate routes to vehicles within a refinement process. This problem was also tackled by Fleischmann within a working paper [6]. Fleischmann attempts to generate a solution using a one-phase algorithm, by integrating a greedy-type heuristic with the need to assign route to vehicles. Using a saving measure with respect to pairs of customers, the feasibility of the assignment of partially constructed routes to vehicles is assessed. The route assignment is achieved by using the bin-packing heuristic *best fit decreasing*, *BFD* (see [8] for details). In this problem bins, items and corresponding weights are defined as vehicles, VRP routes and driver time required to service routes, respectively. A two phase approach was proposed by Taillard et al. [14]. A set of VRP solutions are constructed from a population of routes generated using the Tabu Search (*TS*) heuristic of Rochat and Taillard [10] before bin-packing is used to allocate routes to vehicles. Golden et al. [7] adopted this approach to solve a similar VRPM using the minimax objective. This is a balancing problem which has a wide applicability. A constructive and improvement heuristic was proposed by Brandao and Mercer [1]. They tackled multiple trips as part of a more extensive problem involving time windows and vehicle fleet mix. Using real test data, results showed that their heuristic produced savings of 20% when compared to the manual schedule. To compare with the benchmark of Taillard et al. [14], Brandao and Mercer [2] modified their heuristic to solve the classical VRPM. Their approach is based on the nearest neighbour rule and the insertion criterion to assign customers to routes within vehicles. This process is repeated until all unrouted customers are inserted. The improvement phase attempts

initially to remove overtime before reducing the routing cost within a *TS* framework using two types of trial moves namely insert and swap.

3. Methodology

This section presents the proposed multi-phase heuristic within an algorithmic structure. Initially both notation and objective function are given.

3.1. Preliminaries

We adopt the following notation when describing elements of the methodology:

T	the maximum regular travel time for a vehicle (driver time restriction)
Q	vehicle carrying capacity
NV	the fleet size, which may be defined a priori or determined using the heuristic
NC	the number of customers to service
NR	the number of routes within a solution
S_{VRP}	a VRP solution described as $S_{\mathrm{VRP}} = \{r_1, \ldots, r_k, \ldots, r_{NR}\}$, where r_k is the kth route
vs_i	a schedule of routes assigned to vehicle i described as $vs_i = \{r_{i,1}, \ldots, r_{i,k}, \ldots, r_{i,NR(i)}\}$, where $\{r_{i,1}, \ldots, r_{i,k}, \ldots, r_{i,NR(i)}\} \subseteq S_{\mathrm{VRP}}$
S	a VRPM solution described as follows $S = \{vs_1, \ldots, vs_k, \ldots, vs_{\mathrm{NV}}\}$
$VL(r_i)$	the vehicle load required to service route i
$DT([i,j])$	the driver time between customers i and j
$DT(r_i)$	the total driver time required to service route i
$DT(vs_i)$	the total driver time required to service schedule i
$DT(S)$	the total driver time required to service solution S
$\triangle(vs_i)$	the total driver time infeasibility for schedule i
$\triangle(S)$	the total driver time infeasibility for solution S

The objective function is given as follows:

$$min\{\pi(S)\},$$

where $\pi(S)$, which refers to the maximum driver overtime for solution S, is defined as follows:

$$\pi(S) = max(0, max\{DT(vs_i) - T\}_{i=1,\ldots,NV}). \tag{1}$$

In the literature, the measure OTR_T is used to describe the overtime requirement. Computed using Eq. (2), OTR_T provides a ratio of the driver time for the driver allocated the most work to T.

$$OTR_T(S) = [max\{DT(vs_i)\}_{i=1,\ldots,NV}]/T. \tag{2}$$

The relationship to π is given as follows and provides an alternative to Eq. (1).

$$\pi(S) = \begin{cases} (OTR_T(S) - 1)T & \text{if } OTR_T(S) \geqslant 1, \\ 0 & \text{otherwise.} \end{cases}$$

Note that $OTR_T(S)$ does not define the total amount of overtime present within solution S but it provides the maximum (worst) overtime used in a given schedule. In some circumstances it may be more practical to refer to the scheduled overtime which is given by $\triangle(S)$, where $\triangle(S) = \sum_{i=1}^{NV} max(0, DT(vs_i) - T)$. As an alternative objective function, the scheduling cost for the VRPM is given in Eq. (3), where factors ϕ and p are unit costs for regular driver time and driver overtime penalty, respectively.

$$C(S) = \sum_{i=1}^{NV} [\phi DT(vs_i) + p \triangle (vs_i)]. \tag{3}$$

Note that when a solution S is allowed to use overtime, the legal overtime restriction need not be violated (say a maximum of 2 h per driver per day).

3.2. The main idea of the multi-phase heuristic

The proposed heuristic integrates the approach used by Taillard et al. and that of Brandao and Mercer inasmuch as route construction and solution improvement are undertaken in a VRP and VRPM environment, respectively. The transition from VRP to VRPM solution is achieved by using a bin-packing process which extends the *BFD* heuristic used by Taillard et al. Motivated by the fact that a more successful bin-packing solution could be constructed when many combinations of route driver times are considered, the proposed heuristic generates a variety of VRP solutions. Moreover, two methods are developed to provide these solutions. These methods, which are covered in detail in Section 4, tend to generate different route structures and therefore increase the possibility of finding a more suitable bin-packing. Fig. 1 outlines the heuristic.

4. VRP solution sampling [phases 1 and 3]

4.1. Saving approach (phase 1)

The VRP solutions are generated using the generalized saving measure defined by Yellow [15], within the template saving heuristic of Clarke and Wright [4]. In this study, we allow routes to be constructed in parallel rather than in sequence and restrict customers considered to the initial and final service orders within a route. The measure attaches a prescribed weight δ (also known as shape parameter) to the driver time component of servicing a pair of customers in sequence, see Eq. (4), where D is the depot and $\Omega_{i,j}$ is the saving measure associated with customers i and j when served in one route instead of two separate routes.

$$\Omega_{i,j} = DT([D,i]) + DT([D,j]) - \delta DT([i,j]). \tag{4}$$

[Phase 1]: *Generate VRP Solution Sample* P_1

- **step 1:** This sample is constructed using a saving approach. Refer to subsection 4.1 for detail.
- **step 2:** Transfer sample to working sample, i.e. set $P = P_1$.

[Phase 2]: *VRPM Construction and Improvement Stage*

- **step 3:** Define a rank h for each $S \in P$ such that $DT(S_h) \leq DT(S_{h+1})$. Set counter $i = 1$.
- **step 4:** Set $S_{VRP} = S_i$ and construct VRPM solution S using the bin-packing process described in subsection 5.
- **step 5:** Improve S using improvement procedures described in subsection 6.
- **step 6:** If $i = 1$ then set $S_{best} = S$ else if $OTR_T(S) < OTR_T(S_{best})$ then $S_{best} = S$.
- **step 6:** If $OTR_T(S) \leq 1$ then record S_{best} and stop.
- **step 7:** If $i < |P|$ then set $i = i + 1$ and return to step 4, else if $P = P_1$ go to phase 3, else record S_{best} and stop.

[Phase 3]: *Generate VRP Solution Sample* P_2

- **step 8:** This sample is constructed by selecting routes from a population pool. Refer to subsection 4.2 for detail.
- **step 9:** Transfer sample to working sample, i.e. set $P = P_2$ and return to phase 2.

Fig. 1. Multi-phase heuristic.

By parameterizing this weight in conjunction with a set of imposed driver time restrictions, a sample of VRP solutions can be generated. The sampling process is outlined in Fig. 2.

In this study, parameter values are prescribed as follows: $\delta \in [\delta_{\min}, \delta_{\max}]$, where $\delta_{\min} = 0.6$ and $\delta_{\max} = 1.4$ with a step size of $\xi = 0.1$. For each value of δ, a set of solution pairs S^1_{δ,T^b} and S^2_{δ,T^b} are generated, where T^b represents an imposed driver time bound. Solution $S^1_{T^b}$ is obtained using the saving heuristic followed by individual route refinement through the familiar edge exchange procedures 2-optimal and 3-optimal. Improvement via customer re-routing was not undertaken since an increase of individual route driver times could affect the bin-packing process. Using $S^1_{T^b}$ as an initial solution within a refinement process, a new solution $S^2_{T^b}$ is possible. In step 6, we chose the refinement modules due to Salhi and Rand [12] as these are available to us, although other heuristics could be used. To obtain further solutions we used not only the value of $T^b = T$, but other values with $T^b \leqslant T$ as described in steps 4 and 8 of Fig. 2. In this

- **step 1:** Set $\delta = \delta_{min}$ and $P_1 = \emptyset$.
- **step 2:** Set counter $k = 1$ and $T_k^b = T$.
- **step 3:** Generate solution S_{VRP} using saving heuristic.
- **step 4:** If $k = 1$ set $DT^* = max\ \{DT(r_i)\}_{i=1,\ldots,NR}$ and $\alpha = DT^*\beta$.
- **step 5:** Improve S_{VRP} using 2-optimal and 3-optimal procedures. Set $S^1_{\delta,T_k^b} = S_{VRP}$.
- **step 6:** Improve S_{VRP} using suitable improvement modules and set $S^2_{\delta,T_k^b} = S_{VRP}$.
- **step 7:** Set $P_1 = P_1 \cup \{S^1_{\delta,T_k^b}, S^2_{\delta,T_k^b}\}$.
- **step 8:** If $k < \mu$ set $T_{k+1}^b = DT^* - k\alpha$, $k = k + 1$ and goto step 3.
- **step 9:** If $\delta < \delta_{max}$ set $\delta = \delta + \xi$ and goto step 2.
- **step 10:** Eliminate any repeated solutions within sample P_1.

Fig. 2. Saving based construction.

study, we set $\alpha = 0.05DT^*$ and the number of incremental shifts $\mu = 4$. Experimentation showed that values of $\mu > 4$ produced inferior solutions in most cases. Once the pair of solutions are constructed they are added to the sample P_1 and the process continues for another value of the saving parameter. Finally, any repeated solutions within the sample are eliminated.

4.2. Route population approach (phase 3)

Although the saving approach generates a sample of VRP solutions, the size is relatively small. To increase the size of the sample significantly a method is proposed that constructs solutions through the selection of routes from a pool. There are many ways of generating a population of routes that satisfy the VRP constraints, repeated applications of a metaheuristic being one of those. We chose a tour partition approach since it enables, with use of data structure, an efficient method of route selection needed to develop VRP solutions. The heuristic is outlined in Fig. 3.

The tour construction forms a sector spread about the depot. Initially, each customer i is measured with respect to the positive angle it makes about the depot, defined by θ_i. By ranking each customer according to the magnitude of this measure, a global tour is created. Each customer i has a defined rank h so that $rc_h = i$. Furthermore,

Generate population of routes: P_R

- **step 1:** Compute θ_i for each customer i. Define a rank h for each i in such a way that $rc_h = i$ and rank $j < l \Leftrightarrow \theta_{rc_j} < \theta_{rc_l}$.
  Set $P_R = \emptyset$ and $P_2 = \emptyset$.

- **step 2:** Repeat process for each rank order i. $r_1 = \{rc_i\}$
  $\vdots$
  $r_k = r_{k-1} \cup \{rc_{k+i-1}\}\ \forall k : 2 \leq k \leq l$,
  where $l = max\{m\} : DT(r_m) \leq T$ and $VL(r_m) \leq Q$.
  Note, where $k + i - 1 > NC \Rightarrow rc_{k+i-1} \equiv rc_{k+i-1-NC}$. Moreover, each r_k is improved via 2-optimal and 3-optimal before partition.
  Set $P_R = P_R \cup \{r_1, \ldots, r_l\}$

Construction of VRP sample: P_2

- **step 3:** Via search tree using data structure and reduction, construct VRP solution sample P_2 using P_R as a basis. Sample P_2 is established as follows: three batches of solutions are collected based on $OTR_T(S)$, $DT(S)$ and $\triangle(S)$. Initially, each batch is filled with the first N_0 solutions constructed. The attributes of each new solution generated from the search tree are compared to those of the weakest solution in each batch. If an improvement is found, the solution replaces the weakest. In this study we set $N_0 = 10$.

Fig. 3. Route population pool construction.

$j < l \Leftrightarrow \theta_{rc_j} < \theta_{rc_l}$ where j and l are any two rank orders. The development of a population of routes P_R is conducted by a partitioning process with respect to a giant tour. Steps 2 and 3 of the population heuristic, given in Fig. 3, outline this process. Each ranked customer rc_i is chosen as a first customer visited within a set of routes. This customer automatically forms a trivial single customer route $r_1 = \{rc_i\}$. A second route $r_2 = \{rc_i, rc_{i+1}\}$ is then produced by increasing the rank by 1 and partitioning. Provided both vehicle capacity and driver time restrictions are not violated, this new route is accepted and the process continues. Note that the use of the edge exchange procedures 2-optimal and 3-optimal can be beneficial especially for routes with several customers. If an improved solution is found, the better solution is then checked for feasibility. When a proposed partition is not feasible, the route is not accepted. At this stage a new customer rc_{i+1} is chosen as the first one to be visited within a new set of routes. This process, which is considered for all customers, is illustrated in Fig. 4. The diagram shows 5 route partitions generated from the selection of rc_i as the first customer visited within each route.

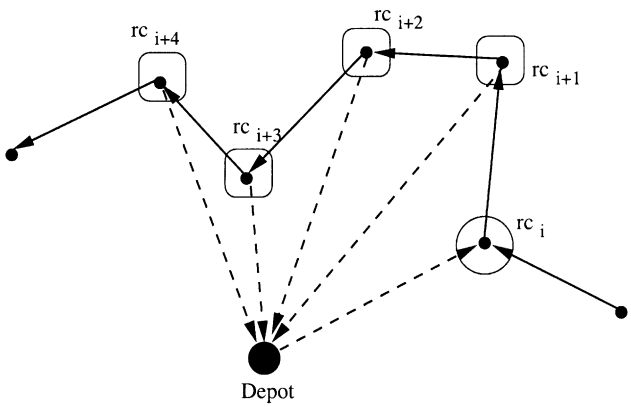

Fig. 4. 5 route partition starting from rc_i and finishing at rc_{i+4}.

4.2.1. A specific data structure

The efficient use of data structure can have a significant impact on reducing CPU time. Structures can hold information calculated or observed at a previous iteration within the heuristic. By installing such a facility, the heuristic can remove the need to recalculate such information at a forthcoming iteration, and therefore unnecessary repeated processing is avoided. The benefit of a good data structure is crucial as it allows the method to be used in larger problems which could not have been solved otherwise, and/or to perform on the same sized problem additional iterations which may improve the solution quality further.

In this approach, VRP solutions are constructed within a search tree environment. Using the population of routes P_{R}, each solution is developed by selecting routes iteratively. To maintain feasibility within solution development, customer duplication is removed. Therefore the selection process requires a method of determining the suitability of a selection choice at each iteration. One possible method would be to undergo an exhaustive checking of customers among those from the selected route and those from the routes already drawn. We propose instead a more efficient method based on route codification. The codification essentially indicates which customers belong to a given route. Each route r_i within population P_{R} of size $NR_{P_{\mathrm{R}}}$, is represented by a pair of unique rank indices $[k_{i1}, k_{i2}]$, where $k_{i1}, k_{i2} \in \{1, \ldots, NC\}$. The values of k_{i1} and k_{i2} are prescribed to be the extremities of the section of the global tour used to define the route. Note that the code also identifies the sector spread within the xy plane. Fig. 5 illustrates a route which has a sector with angle $\theta_{rc_i} - \theta_{rc_j}$. The code for such a route is $[i, j]$.

Duplication between this route and one defined by $[k, l]$, is tested as follows: if $j \geqslant k \geqslant i$ or $j \geqslant l \geqslant i$ then the routes have customers in common. For example, a route with code [6,1] has two customers in common with the route [7,15], these being rc_6 and rc_7. The heuristic uses a selection strategy based on sequential inclusion of customers within the global tour. In other words, once an initial selection r_i is made, further

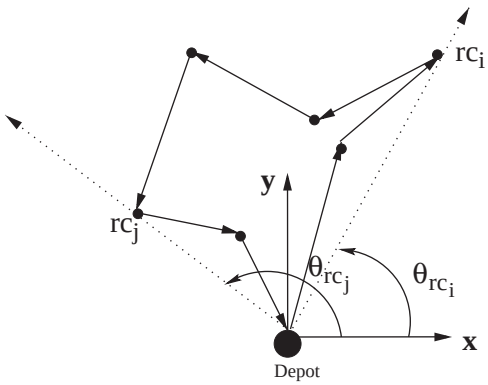

Fig. 5. Route code.

selections are restricted to those routes $r_j : k_{j1} = k_{i2} + 1$. The process continues until a complete solution is formed. To reduce the number of computational comparisons the population of routes is ordered with respect to the indices, giving a priority to $k_{i,1}$. Each choice of route selected at a given development stage is explored and, where a solution is possible, it is added to the sample P_2. To avoid the formation of duplicate solutions, the choice of initial route selection is restricted as follows:

$$r_i \in \{r_1, \ldots, r_n\} \quad \text{where } n = min\{m : k_{m1} = k_{NR_{P_R}2} + 1\}.$$

Illustrative example: Consider the following simple example, where $NC = 3$ and route population P_R, given as follows:

$$P_R = \{[1,1],[1,2],[2,2],[2,3],[3,3],[3,1],[3,2]\}.$$

The sample of VRP solutions is given by $P_2 = \{S_1 = \{[1,1],[2,2],[3,3]\}, S_2 = \{[1,1],[2,3]\}, S_3 = \{[1,2],[3,3]\}, S_4 = \{[2,2],[3,1]\}, S_5 = \{[3,2]\}\}$. The selection strategy is assisted by a data structure. Each route is labelled with the index pair defining the routes which limit the next selection. The structure is generalized in Table 1. The first and second columns represent the index of a route belonging to the population and associated coding, respectively. The new labelling for each route r_i is represented by i_1 and i_2. For example, if route r_1 is selected the next choice is taken from the set $\{r_{k_1+1}, \ldots, r_{k_1+k_2-1}\}$.

4.2.2. Reduction techniques

A drawback of the selection process is the construction of a significant number of solutions which can be very poor. For instance, a solution is generated which consists of NC routes each of which is single. Whereas a typical VRP constructive method requires a greedy-type measure to direct the development, the proposed heuristic has no such measure due to the fact that the VRP solution is only a basis for transformation. We address this problem, with the use of two reduction techniques. They guide the construction by restricting the selection based on characteristics of a typical solution S_s generated from the saving heuristic.

Table 1
Data structure for route selection

Index $i : r_i \in P_R$	$[k_{i1}, k_{i2}]$	Index i_1	Index i_2
1	$[1,1]$	k_1+1	k_1+k_2-1
2	$[1,2]$	k_1+k_2	$k_1+k_2-1+k_3-2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
k_1	$[1,k_1]$	$\vdots$	$\vdots$
k_1+1	$[2,2]$	$\vdots$	$\vdots$
k_1+2	$[2,3]$	$\vdots$	$\vdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
k_1+k_2-1	$[2,k_2]$	$\vdots$	$\vdots$
k_1+k_2	$[3,3]$	$\vdots$	$\vdots$
k_1+k_2+1	$[3,4]$	$\vdots$	$\vdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$k_1+k_2-1+k_3-2$	$[3,k_3]$	$\vdots$	$\vdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Technique (i): The first technique consists of a bound NR_b with respect to the number of routes within a solution. At each stage of development, the number of routes drawn is compared against the bound. If the bound is violated, the previous selection is replaced and another is considered. When no other choices are available, again the previous selection is replaced and the process continues. The restriction is given as follows:

$$NR \leqslant NR_b \quad \text{where } NR_b = [1.1NR_s].$$

Technique (ii): The second reduction technique is a restriction with respect to the number of customers within routes. Again using solution S_s, the customer range is partitioned into n segments. By attaching a restriction for each segment range, a criterion for route selection is established. The technique is adaptive, since the previous selection choices determine the choices made at future selection stages. In this study, we set $n = 6$ and prescribe the partitions ranges as follows, where $[x_{i-1}, x_i)$ represents segment i and the boundaries $x_0 = 1$ and $x_6 = NC$.

$$[x_{i-1}, x_i) \quad i = 1, \ldots, 6,$$

$$x_i = [\bar{x} + (i-3)\sigma] \quad i = 1, \ldots, 5,$$

$$\bar{x} = NC/NR_s,$$

$$\sigma = \sqrt{\sum_{i=1}^{NR_s} \frac{(NC_s(i) - \bar{x})^2}{NR_s - 1}}.$$

A percentage α_i of NR_b is attached to each range i. Each attachment provides a bound on the number of routes selected. In this study we set $\alpha_1 = 5$, $\alpha_2 = 50$, $\alpha_3 = 75$, $\alpha_4 = 75$, $\alpha_5 = 50$, $\alpha_6 = 5$. Note $\sum_{i=1}^{6} \alpha_i > 100$, since this allows the development of solutions which differ significantly from S_s.

5. Bin-packing: The allocation of routes to vehicles (phase 2, step 4)

The transformation from S_{VRP} to S, where S is a solution to the VRPM, is achieved by solving several bin-packing problems, where items and weights are VRP routes and corresponding driver times, respectively. For a given fleet size restriction NV_{max}, a series of bin sizes are considered. NV_{max} can be prescribed by the user or computed as follows:

$$NV_{max} = \begin{cases} \left[\sum_{i=1}^{NR} DT(r_i)/T\right] + 1 & \text{if } \sum_{i=1}^{NR} DT(r_i)/T - \left[\sum_{i=1}^{NR} DT(r_i)/T\right] > 0.6, \\ \left[\sum_{i=1}^{NR} DT(r_i)/T\right] & \text{otherwise.} \end{cases}$$

The value 0.6 is used to provide a stricter condition than the nearest integer to $\sum_{i=1}^{NR} DT(r_i)/T$, which would be expressed by 0.5. In other words, this is introduced to tighten the bound. The heuristic is outlined in Fig. 6. Based on the *BFD* algorithm, used by Taillard et al., the process assigns routes according to minimal residual driver time for a subset of vehicles. A vehicle is selected to form an initial subset. This vehicle is then assigned routes in order of driver time magnitude without violating T. At this point, another vehicle is added to the subset and routes are then allocated to the vehicle with minimal residual driver time. The process continues until the fleet restriction NV_{max} is violated where upon the remaining routes are assigned to vehicles allowing the violation of the driver time restriction T. Attempting to minimize the amount of infeasibility, the route with the maximum driver time is selected from those remaining and assigned to the vehicle with the least amount of work allocated. This process continues until all routes have been assigned. Step 4 of the heuristic considers an improvement to the bin-packing through the reassignment of routes to vehicles. We propose a suite of 3 modules which attempt to decrease driver time infeasibility. The modules, which consider the rescheduling of single routes within two and three vehicle frameworks, are implemented within a simple composite multi-level structure. The imposed bin sizes are prescribed using a bisection approach with a restricted number of iterations k_{max}. In this study we set $k_{max} = 5$, although with larger problems, a slightly higher value would be more suitable. A bin size of T is used to generate an initial solution. If the solution requires no overtime the process stops otherwise a new driver bound is prescribed as defined by the bisection in step 2. At each iteration k,

- **step 1:** Set counter $k = 1$
- **step 2:** If $k = 1$ set $T_k^b = T$ else set $T_k^b = (T^u + T^l)/2$.
- **step 3:** Generate VRPM solution S via bin-packing
  $r_j \in S_{VRP}\ \forall j$ using modified BFD heuristic:
  [fleet restriction NV_{max}, bin size T_k^b].
  If $k = 1$ set $S^b = S$.
  If $OTR_T(S) \leq 1$ set $S^b = S$ and stop.
- **step 4:** Attempt to improve S using the route reassignment module suite.
  If $OTR_T(S) \leq 1$ set $S^b = S$ and stop
  else if $OTR_T(S) \leq OTR_T(S^b)$ set $S^b = S$.
- **step 5:** If $k = 1$ set $T^l = T_k^b, T^u = max\{DT(vs_j)\}_{j=1,\ldots,NV}$
  $k = k + 1$ and goto step 2.
- **step 6:** If $OTR_{T_k^b}(S) \leq 1$ set $T^u = T_k^b$ else set $T^l = T_k^b$.
- **step 7:** If $k = k_{max}$ stop else set $k = k + 1$ and goto step 2.

Fig. 6. Several bin-packing applications.

the solution generated S is compared against the best found thus far S^{b}. If S is an improvement then $S^{\mathrm{b}} = S$.

6. Improvement modules (phase 2, step 5)

This section describes a set of improvement modules designed to address both the objective function given in Section 3.1 and the reduction in the solution cost. Consisting of 5 module groups (known as *Meiosis*, *VRP_Partition*, *Donate*, *Exchange* and *Donate_Exchange*) a composite multi-level structure is chosen for module selection. The structure is outlined in Fig. 7. This multi-level approach has been successfully adapted for the multi-depot vehicle routing problem by Salhi and Sari [13]. The following subsections outline the characteristics of each module group.

6.1. Module: Meiosis

A phenomenon in biology, Meiosis is the name given to the process where a cell divides to create two new unique cells. Borrowing this terminology, route meiosis, which is the transformation used in module *Meiosis*, describes the division of a route to generate two new tours. This concept is also known in the OR literature as 'route split' (see [12]). The new routes are formed by partitioning a tour about a pivotal customer, maintaining partial route structure and coupling the end sections to the depot. In this sense, a pivotal customer defines a partition for a given route. By improving

- **step 1:** Use the module *Meiosis*.
- **step 2:** Use the module *VRP_Partition*.
- **step 3:** Construct the Neighbourhood $N(S)$ using module *Donate*. If $|N(S)| > 0$ then select $S^* : C_p(S^*) = min\{C_p(S')\}_{S' \in N(S)}$, and set $S = S^*$ and repeat step, else continue.
- **step 4:** Construct the Neighbourhood $N(S)$ using module *Exchange*. If $|N(S)| > 0$ then select $S^* : C_p(S^*) = min\{C_p(S')\}_{S' \in N(S)}$, set $S = S^*$ and goto step 3, else continue.
- **step 5:** Construct the Neighbourhood $N(S)$ using module *Donate_Exchange*. If $|N(S)| > 0$ then select $S^* : C_p(S^*) = min\{C_p(S')\}_{S' \in N(S)}$, set $S = S^*$ and goto step 3, else stop.

Fig. 7. Improvement modules.

each route using familiar edge exchange procedures, such as 2-optimal and 3-optimal, and reallocating one of the newly formed routes to an alternative vehicle, schedule overtime can be reduced. The meiotic transformation is expressed as follows, where r_1, r_2 and $r_{i,j} \in vs_i$ are the two newly created routes and the original route, respectively. The choice of the pivot customer is determined by the quality of the new solution after route reallocation.

$$r_{i,j} \rightarrow r_1, r_2.$$

In this context, either route can be relocated to a schedule vs_k, $k \neq i$. For instance $r'_{i,j} = r_1$ and $r'_{k,NR'(k)} = r_2$, where $r'_{i,j}$ and $r'_{k,NR'(k)}$ are the newly scheduled routes. Note that the number of routes allocated to the kth vehicle schedule has increased by one, i.e. $NR'(k) = NR(k) + 1$. Furthermore, in most cases $DT(r_1) + DT(r_2) \geqslant DT(r_{i,j})$, i.e. the transfer of time between drivers generates the need for more work. This fact is tolerated since the module is used to address the situation whereby there is a significant variation of driver time about T.

6.1.1. Neighbourhood and selection strategy

For a given solution S and newly generated routes $r'_{i,j}$ and $r'_{k,NR'(k)}$, a new solution S' is defined using the following expressions, where vs'_i and vs'_k are the associated new vehicle schedules.

$$S' = ((S \setminus \{vs_i\}) \setminus \{vs_k\}) \cup \{vs'_i\} \cup \{vs'_k\},$$

$$vs'_i = (vs_i \setminus \{r_{i,j}\}) \cup \{r'_{i,j}\},$$

$$vs'_k = vs_k \cup \{r'_{k,NR'(k)}\}.$$

- **step 1:** Generate $N(S)_2$ s.t. $\dot{S} \in N(S)_2$ if $OTR_T(\dot{S}) < OTR_T(S)$
- **step 2:** Select $N(S)_3 \subseteq N(S)_2$ such that $\ddot{S} \in N(S)_3$ if $OTR_T(\ddot{S}) = min\{OTR_T(\dot{S})\}_{\dot{S} \in N(S)_2}$
- **step 3:** If $|N(S)_3| = 0$ stop, else choose $S' = S^* : OTR_T(S^*(vs'_i, vs'_k)) = min\{OTR_T(\ddot{S}(vs'_i, vs'_k))\}_{\ddot{S} \in N(S)_3}$, and set $S = S'$ and goto step 1.

Fig. 8. Selection strategy: *Meiosis*.

Technically a large neighbourhood $N(S)_1$ can be generated by considering all routes as candidates for meiosis and for each partition, all combinations of rescheduling to vehicles. The cardinality of such a neighbourhood is as follows:

$$|N(S)_1| = \sum_{i=1}^{NR} NC(i) \times 2(NV - 1).$$

The module uses a minimax objective function with respect to driver time assigned to vehicles, i.e. $min\{OTR_T(S')\}_{S' \in N(S)_1}$. This function allows a smaller neighbourhood to be considered. Strictly non-improving solutions can be avoided by restricting candidate meiosis routes to belong to the vehicle scheduled the most work. This condition is expressed as follows:

$$r_{i,j} \in vs_i : vs_i = max\{DT(vs_j)\}_{j=1,\ldots,NV}.$$

Furthermore, we can remove all other non-improving solutions to remain with a new neighbourhood $N(S)_2$ which has cardinality bound as follows:

$$0 \leqslant |N(S)_2| \leqslant \sum_{j=1}^{NR(i)} NC(j) \times 2(NV - 1).$$

The strategy used to select which solution to accept is outlined in Fig. 8. Here a new measure $OTR_T(S(vs'_i, vs'_k))$ is introduced, and is defined as follows:

$$OTR_T(S(vs'_i, vs'_k)) = max(DT(vs'_i), DT(vs'_k))/T,$$

$$DT(vs'_i) = DT(vs_i) - DT(r_{i,j}) + DT(r'_{i,j}),$$

$$DT(vs'_k) = DT(vs_k) + DT(r'_{k,NR'(k)}).$$

This caters for the likely outcome of more than one solution having the minimum objective value. It refers to the maximum driver time scheduled with respect to the new schedules vs'_i and vs'_k. Step 4 enables a selection based upon a minimum of such a measure.

6.2. Module: *VRP_Partition*

Each subset of routes allocated to a vehicle can be viewed as a VRP solution. This enables the use of a VRP heuristic to improve the drivers schedule. In this study, we used the improvement modules of Salhi and Rand [12] as briefly described in the following subsection. Module *VRP_Partition* describes the use of this set within the context of partitioning the VRPM solution. The partition is made according to vehicle schedule. In other words a partition refers to a subset of routes characterized by the vehicle schedule they are allocated to. Each subset of routes is then improved using the module suite of Salhi and Rand.

The idea can be expressed more formally as follows: Let S be a VRPM solution, where $S = \{vs_1, \ldots, vs_{\mathrm{NV}}\}$ and $vs_i = \{r_{i,1}, \ldots, r_{i,NR(i)}\}$. We can then partition S into NV small VRP solutions $\{S^i_{\mathrm{VRP}}\}_{i=1,\ldots,\mathrm{NV}}$, by defining $S^i_{\mathrm{VRP}} = vs_i$. Implementing module *VRP_Partition* on each VRP solution S^i_{VRP} produces a new VRPM solution S' where $S' = \{vs'_1, \ldots, vs'_{\mathrm{NV}}\} = \{(S^i_{\mathrm{VRP}})', \ldots, (S^{\mathrm{NV}}_{\mathrm{VRP}})'\}$ and $(S^i_{\mathrm{VRP}})'$ is the solution generated from S^i_{VRP} using module *VRP_Partition*.

The success of this module is dependent upon the size of the VRP solutions considered. For example a VRPM problem where $NC = 50$ and $NV = 5$, giving an average of 10 customers for each *VRP* solution, suggests little room for improvement. Nevertheless, the module is quick and therefore justifies its inclusion at this stage of the heuristic. Furthermore, when the module *Meisois* has been successfully implemented the success of *VRP_Partition* is increased significantly.

6.2.1. VRP improvement routines

This suite of refinements comprises the following; initially a module OPT, which consists of 2-optimal and 3-optimal procedures, is used for each route. This is followed by the module COMBINE, which assesses the feasibility of joining pairs of routes. The solution of each successful combination is accepted. After eliminating route crossing using OPT, each solution enters a customer reallocation phase. A single customer reallocation module SHIFT, which reallocates customers between routes, is followed by the module PERTURB which considers three routes simultaneously. Here a route accepts a customer but also relinquishes a customer, hence the modification of three routes. The entire process is repeated until there is no improvement in each of these modules.

6.3. Single customer reallocation

Here, we briefly describe the module transformations: *Donate*, *Exchange* and *Donate_Exchange*. Each module attempts to improve the solution through the route rescheduling of customers. Although in most cases a customer is relocated to an alternative vehicle schedule, rescheduling to other routes within its existing schedule is also considered.

Module: *Donate*: The module *Donate* is an adaptation of the familiar insertion move, whereby a customer, β say, is donated to a particular route $r_{k,l} \in S \setminus \{r_{i,j}\}$ where $\beta \in r_{i,j}$.

The module transformation, which requires the modification of 2 routes, is expressed as follows, where $r'_{i,j}$ and $r'_{k,l}$ are the routes created following the customer reschedule:

$$r_{i,j} \to r'_{i,j},$$

$$r_{k,l} \to r'_{k,l}.$$

Module: *Exchange*: This module, like that of module *Donate*, requires the modification of 2 routes to implement the schedule change. Module *Exchange* is an adaptation of the exchange move, whereby a pair of routes each exchange a customer. The transformation is expressed as follows, where $r^t_{i,j}$ and $r^t_{k,l}$ represent the partially transformed routes created by removing corresponding customers β and ξ. This intermediate stage is necessary, so that the new links created from customer removal can be considered as candidate insertion locations.

$$r_{i,j} \to r^t_{i,j} \to r'_{i,j},$$

$$r_{k,l} \to r^t_{k,l} \to r'_{k,l}.$$

Module: *Donate_Exchange*: This module *Donate_Exchange* provides an integration of the modules described previously. Involving 3 routes, 2 routes donate customers but one of them also accepts a customer. The operation reschedules 2 customers, where (β, ξ) represent a customer pair rescheduled between the 3 routes $r_{i,j}$, $r_{k,l}$ and $r_{m,n}$. The transformation is expressed as follows, where $r^t_{k,l}$ represents the partially transformed route created by removing customer ξ. This is necessary to allow the possibility of inserting customer β at new link positions.

$$r_{i,j} \to r'_{i,j},$$

$$r_{k,l} \to r^t_{k,l} \to r'_{k,l},$$

$$r_{m,n} \to r'_{m,n}.$$

6.4. Other technical issues

In this section, we present three key factors that enhanced the implementation of this heuristic.

6.4.1. Effect of empty routes

A weakness of customer reallocation, within a multiple trip framework, is when a vehicle schedule has spare driver time capacity but minimal or no spare vehicle load capacity. In other words, potentially good candidate solutions, for customer reallocation, are not considered because vehicle load capacity would be exceeded. We address this problem by providing each vehicle schedule vs_i, $i = 1, \ldots, NV$ with an empty route. This can be expressed using Eq. (5). Note that each schedule increases the number of routes it services by 1, i.e. $NR'(i) = NR(i) + 1 \ \forall i = 1, \ldots, NV$, where $NR'(i)$ is the new

number of routes assigned to the ith schedule.

$$r_{i,NR'(i)} = \emptyset \quad \forall i = 1, \ldots, NV. \tag{5}$$

By scheduling customers to empty routes, idle driver time can be addressed, without the restrictions of vehicle capacity violation. This advantage is an important consideration for the VRPM especially when schedules contain only a few routes.

6.4.2. A guided neighbourhood generation

For a given solution S and a given refinement module, a neighbourhood of candidate transformation solutions $N(S)$ is generated. $N(S)$ is made up of all those solutions which are feasible with respect to vehicle capacity, and which improve on the current value $\pi(S)$. The question is to find which solution to select from $N(S)$. If we use the objective function as defined by Eq. (1) then the heuristic is restricting transformations to those which minimizes $\pi(S)$. Although this is our primary goal, the search can be restrictive, since suitable candidates with respect to the scheduling costs are not considered. We address this problem by using a variable penalty function, C_p, expressed in terms of driver overtime. We assign a higher penalty cost to driver schedules which require more overtime. We do this by partitioning the region $\pi(S)$ into n segments and assigning each sector a unique penalty factor. The penalty function is defined as follows, where each segment is of size $\xi = \pi(S)/n$.

$$C_p(S) = \sum_{i=1}^{NV} [\phi DT(vs_i) + p_i \triangle (vs_i)],$$

where

$$p_i = \begin{cases} \mu_1 & \text{if } 0 < \triangle (vs_i) \leqslant \xi, \\ \mu_2 & \text{if } \xi < \triangle (vs_i) \leqslant 2\xi, \\ \vdots & \vdots \\ \mu_j & \text{if } (j-1)\xi < \triangle (vs_i) \leqslant j\xi, \\ \vdots & \vdots \\ \mu_n & \text{if } (n-1)\xi < \triangle (vs_i) \leqslant n\xi, \\ 0 & \text{otherwise.} \end{cases}$$

Note $\mu_n > \mu_{n-1} > \cdots > \mu_1 > 0$, i.e. higher schedule overtime implies higher cost. In this study, we set n=5, $\mu_i = \mu_{i-1} + 1$ $\forall i > 1$ with $\mu_1 = 1$. Although other parameter values could be used, we observed that the prescription of large values was too restrictive. Using the penalty function, the choice of which solution S' to select as the new solution S is governed by the following objective function:

$$min\{C_p(S')\}_{S' \in N(S)}.$$

This new objective function addresses both solution cost and overtime requirement π. It encourages transformations which address $\pi(S)$ but does not disregard good scheduling moves involving other driver schedules.

6.4.3. Use of an existing data structure

The proposed heuristic makes use of a data structure similar to the one adopted in [9]. The structure assists with the calculations required to establish customer reallocation. For each module, a neighbourhood is constructed to take into account the use of this structure.

7. Computational experience

The heuristic was tested against the benchmarks achieved by Taillard et al. [14] (TLG). The data sets used are generated from a set of 9 VRP base problems, with unrestricted driver time, taken from Christofides et al. [3] and Fisher [5], respectively. By imposing a series of driver time restrictions, a set of subproblems are created for each base problem. In total, 104 subproblems were generated. The driver restrictions are based on the VRP solution obtained by Rochat and Taillard [10]. The fleet size used for each sub-problem defines the value $NV_{\max}$ used in the heuristic. Moreover, overtime is not permitted. All the problems are Euclidean and it is assumed that the driver time required to travel between each pair of customers equals the corresponding distance. The heuristic is also compared against the results achieved by Brandao and Mercer [2] (BM) who used the same data sets. The heuristic algorithms were coded in Fortran 90 and executed on a Ultra Enterprise 450 dual processor at 300 MHz, although the dual aspect of the processor was not considered.

The appendix provides a comprehensive set of results. For each instance tested, the measure OTR_T is given. The significance of the maximum overtime $\pi(S)$ can be measured as the percentage of the normal driver time T. Such a measure is referred to as $\alpha(S)$ and is calculated as follows:

$$\alpha(S) = (\pi(S)/T) \times 100, \tag{6}$$

$$= max((OTR_T(S) - 1) \times 100, 0). \tag{7}$$

The following statistics help to assess the performance of the heuristic, where S_i^1 and S_i^2 represent solutions from the tested heuristic and benchmark, respectively, for instance i ($i = 1, N_{\mathrm{I}}$), and N_{I} is the number of instances considered.

$$\bar{\alpha} = \frac{1}{N_{\mathrm{I}}} \sum_{i=1}^{N_{\mathrm{I}}} \alpha(S_i),$$

$$\zeta = \frac{100}{N_{\mathrm{I}}} \sum_{i=1}^{N_{\mathrm{I}}} \left\{ \frac{\pi(S_i^1)}{\pi(S_i^2)} - 1 \right\}.$$

Table 2
Significance of π

Statistic	*PM*	*TLG*	*BM*
$\bar{\alpha}$	0.72	0.86	0.53

Table 3
Comparison of *PM* against *TLG* and *BM*

Statistic	*TLG*	*BM*
ζ	29.59	−25.27

- $\bar{\alpha}$ is the average driver time restriction as a % of normal driver time T.
- ζ is the average % decrease in driver restriction compared to given benchmark.

The results from OTR_T show that a solution requiring no overtime was obtained in most instances. Closer inspection reveals that the instances where overtime is needed a small value of T was prescribed. This relationship between T and OTR_T is found throughout the data sets tested, see the appendix. If we compare the number of instances where solutions were found which required no overtime, then *PM* produces fewer compared to both benchmarks. Results found that 73.08% required no overtime compared to 77.88% achieved by *TLG* and 85.58% from *BM*. Table 2 provides the results using statistic $\bar{\alpha}$. Subscripts 1 and 2 refer to the results obtained if the number of instances is restricted to those where overtime was needed with respect to *TLG* and *BM*, respectively. The average overtime restriction for *PM* is 0.72% which compares to 0.86% and 0.53% for benchmarks *TLG* and *BM*, respectively. These results suggest that although the heuristic fails to generate as many overtime free solutions, where there is the presence of overtime within a solutions, *PM* generates a competitive overtime restriction π. Table 3 provides the results for statistic ζ: this table describes the average percentage decrease in required overtime prescription $\pi(S)$ when compared to a given benchmark. In each case the instances are restricted to those from the particular benchmark where overtime is present. When comparing against *TLG* we find that the proposed heuristic performs well, where the average overtime is 29.59% lower, whereas comparing against *BM* the average is 25.27% higher. If we consider the instances where the proposed heuristic generated a solution requiring overtime and both benchmarks required no overtime, the average overtime restriction was only 1.24%. This fact suggests that often the proposed heuristic is generally extremely close to feasibility in terms of overtime use. If we consider the number of instances which are an improvement, equivalent and worse than a given benchmark we obtain the following results: $(19, 68, 17)$ and $(8, 74, 22)$ with respect to *TLG* and *BM*, respectively. These results are promising especially with respect to *TLG* where a higher number of better solutions is obtained, i.e. 19 against

Table 4
Average CPU time

BP	CPU_{PM}	CPU_{TLG}	CPU_{BM}
$C1_{18}$	1.8	5	2.5
$C2_{14}$	5.5	7	5
$C3_{12}$	13.8	24	10
$C4_{16}$	16.4	51	25
$C5_{20}$	40.9	66	62.5
$C11_{10}$	40.5	45	25
$C12_{12}$	2	23	10
$F71_{6}$	4.3	26	2.5
$F134_{6}$	13.5	75	80

17. The results with respect to *CPU* time in minutes are given in Table 4. In each case the subscript indicates the number of subproblems for each base problem *BP*. As the results suggest, the heuristic competes quite favorably with both benchmarks although the importance of this is minor since the heuristic was tested on a faster machine.

8. Conclusions

A multi-phase constructive heuristic has been proposed and tested to solve the vehicle routing problem with multiple trips. The main phases consist of two approaches to VRP construction, the allocation of routes to vehicles and improvement modules. A data structure and two reduction techniques are embedded into the search to speed up the process without affecting the solution quality. Empirical testing is used to assess the performance of this technique with encouraging results. As for future investigations, this approach can be adopted to solve the vehicle fleet mix with multiple trips without too many modifications. A possible extension will be to integrate a genetic algorithm with the population of solutions already generated. The authors are currently investigating this issue. From a practical view point, a scenario analysis could be carried out to determine which is the most economical fleet size given the different solutions for each level of overtime.

Acknowledgements

The authors are grateful to both referees for their constructive comments which improved both the content and the presentation of the paper. We thank the School of

Table 5
Results: $C1$–$C3$

BP	T	NV	OTR_T^1	OTR_T^2	OTR_T^3
$C1$	551	1	0.990	$\leqslant 1$	$\leqslant 1$
	275	2	1.004	$\leqslant 1$	$\leqslant 1$
	184	3	1.026	1.115	1.041
	138	4	1.085	1.027	1.027
	577	1	0.997	$\leqslant 1$	$\leqslant 1$
	289	2	0.981	$\leqslant 1$	$\leqslant 1$
	192	3	1.000	1.050	$\leqslant 1$
	144	4	1.031	$\leqslant 1$	$\leqslant 1$
$C2$	877	1	0.991	$\leqslant 1$	$\leqslant 1$
	439	2	0.998	$\leqslant 1$	$\leqslant 1$
	292	3	0.994	$\leqslant 1$	$\leqslant 1$
	219	4	0.998	$\leqslant 1$	$\leqslant 1$
	175	5	0.999	$\leqslant 1$	$\leqslant 1$
	146	6	1.019	1.032	1.031
	125	7	1.064	1.073	1.088
	919	1	0.945	$\leqslant 1$	$\leqslant 1$
	459	2	0.987	$\leqslant 1$	$\leqslant 1$
	306	3	0.987	$\leqslant 1$	$\leqslant 1$
	230	4	0.957	$\leqslant 1$	$\leqslant 1$
	184	5	0.995	$\leqslant 1$	$\leqslant 1$
	153	6	0.994	$\leqslant 1$	$\leqslant 1$
	131	7	1.009	1.023	$\leqslant 1$
$C3$	867	1	0.998	$\leqslant 1$	$\leqslant 1$
	434	2	0.981	$\leqslant 1$	$\leqslant 1$
	289	3	0.997	$\leqslant 1$	$\leqslant 1$
	217	4	0.991	$\leqslant 1$	$\leqslant 1$
	173	5	1.052	1.062	$\leqslant 1$
	145	6	1.001	1.032	1.003
	909	1	0.981	$\leqslant 1$	$\leqslant 1$
	454	2	0.999	$\leqslant 1$	$\leqslant 1$
	303	3	0.999	$\leqslant 1$	$\leqslant 1$
	227	4	0.996	$\leqslant 1$	$\leqslant 1$
	182	5	0.999	1.010	$\leqslant 1$
	151	6	0.996	1.012	$\leqslant 1$

Mathematics and Statistics of the University of Birmingham for the sponsorship of the first author.

Appendix A. Detailed computational results

The following series of Tables 5–7 provide the results of the proposed methodology and those results published from the literature. The table columns are given as

Table 6
Results: $C4$–$C5$

BP	T	NV	OTR_T^1	OTR_T^2	OTR_T^3
$C4$	1080	1	0.999	$\leqslant 1$	$\leqslant 1$
	540	2	0.998	$\leqslant 1$	$\leqslant 1$
	360	3	0.995	$\leqslant 1$	$\leqslant 1$
	270	4	1.005	$\leqslant 1$	$\leqslant 1$
	216	5	0.999	$\leqslant 1$	$\leqslant 1$
	180	6	1.010	$\leqslant 1$	$\leqslant 1$
	154	7	1.072	1.033	1.071
	134	8	1.058	1.075	1.031
	1131	1	0.999	$\leqslant 1$	$\leqslant 1$
	566	2	1.000	$\leqslant 1$	$\leqslant 1$
	377	3	0.998	$\leqslant 1$	$\leqslant 1$
	283	4	0.998	$\leqslant 1$	$\leqslant 1$
	226	5	0.994	$\leqslant 1$	$\leqslant 1$
	189	6	0.980	$\leqslant 1$	$\leqslant 1$
	162	7	1.005	1.010	$\leqslant 1$
	141	8	0.993	1.029	$\leqslant 1$
$C5$	1356	1	0.999	$\leqslant 1$	$\leqslant 1$
	678	2	1.000	$\leqslant 1$	$\leqslant 1$
	452	3	0.996	$\leqslant 1$	$\leqslant 1$
	339	4	0.998	$\leqslant 1$	$\leqslant 1$
	271	5	1.007	$\leqslant 1$	$\leqslant 1$
	226	6	1.000	$\leqslant 1$	$\leqslant 1$
	194	7	1.008	$\leqslant 1$	$\leqslant 1$
	170	8	1.015	$\leqslant 1$	$\leqslant 1$
	151	9	1.024	$\leqslant 1$	1.056
	136	10	1.064	1.024	1.051
	1421	1	0.993	$\leqslant 1$	$\leqslant 1$
	710	2	0.995	$\leqslant 1$	$\leqslant 1$
	474	3	1.000	$\leqslant 1$	$\leqslant 1$
	355	4	0.995	$\leqslant 1$	$\leqslant 1$
	284	5	0.992	$\leqslant 1$	$\leqslant 1$
	237	6	0.997	$\leqslant 1$	$\leqslant 1$
	203	7	0.998	$\leqslant 1$	$\leqslant 1$
	178	8	1.000	$\leqslant 1$	$\leqslant 1$
	158	9	0.998	$\leqslant 1$	$\leqslant 1$
	142	10	1.018	$\leqslant 1$	$\leqslant 1$

follows:

BP	Base problem tested, where Ci and Fi refer to ith dataset taken from Christofides et al. [3] and Fisher [5], respectively
T	the maximum travel time for a vehicle
NV	the fleet size required
OTR_T	the ratio of the driver allocated the most work to T
Superscripts 1, 2 and 3	Refer to results obtained from PM, TLG and BM, respectively

Table 7
Results: $C11$–$C12, F71, F134$

BP	T	NV	OTR_T^1	OTR_T^2	OTR_T^3
$C11$	1094	1	0.996	$\leqslant 1$	$\leqslant 1$
	547	2	1.006	$\leqslant 1$	$\leqslant 1$
	365	3	1.006	$\leqslant 1$	$\leqslant 1$
	274	4	1.052	1.020	1.011
	219	5	1.037	$\leqslant 1$	$\leqslant 1$
	1146	1	1.000	$\leqslant 1$	$\leqslant 1$
	573	2	0.991	$\leqslant 1$	$\leqslant 1$
	382	3	0.970	$\leqslant 1$	$\leqslant 1$
	287	4	1.002	$\leqslant 1$	$\leqslant 1$
	229	5	0.999	$\leqslant 1$	$\leqslant 1$
$C12$	861	1	0.993	$\leqslant 1$	$\leqslant 1$
	430	2	0.988	$\leqslant 1$	$\leqslant 1$
	287	3	0.974	$\leqslant 1$	$\leqslant 1$
	215	4	0.991	$\leqslant 1$	1.012
	172	5	1.000	1.050	1.036
	143	6	1.029	1.064	1.072
	902	1	0.948	$\leqslant 1$	$\leqslant 1$
	451	2	0.958	$\leqslant 1$	$\leqslant 1$
	301	3	0.950	$\leqslant 1$	$\leqslant 1$
	225	4	0.999	$\leqslant 1$	$\leqslant 1$
	180	5	0.986	1.003	$\leqslant 1$
	150	6	0.999	1.014	$\leqslant 1$
$F71$	254	1	1.000	$\leqslant 1$	$\leqslant 1$
	127	2	1.020	1.031	1.011
	85	3	1.020	1.075	1.011
	266	1	0.997	$\leqslant 1$	$\leqslant 1$
	133	2	0.979	$\leqslant 1$	$\leqslant 1$
	88	3	0.998	1.027	$\leqslant 1$
$F134$	1221	1	0.980	$\leqslant 1$	$\leqslant 1$
	611	2	0.996	$\leqslant 1$	$\leqslant 1$
	407	3	0.988	$\leqslant 1$	$\leqslant 1$
	1279	1	0.961	$\leqslant 1$	$\leqslant 1$
	640	2	0.996	$\leqslant 1$	$\leqslant 1$
	426	3	0.994	$\leqslant 1$	$\leqslant 1$

References

[1] J. Brandao, A. Mercer, A tabu search algorithm for the multi-trip vehicle routing and scheduling problem, Eur. J. Oper. Res. 100 (1997) 180–191.
[2] J. Brandao, A. Mercer, The multi-trip vehicle routing problem, J. Oper. Res. Soc. 49 (1998) 799–805.
[3] N. Christofides, P. Mingozzi, P. Toth, C. Sandi (Eds.), The vehicle routing problem, in: Combinatorial Optimization, Wiley, Chichester, 1979.
[4] G. Clarke, J.W. Wright, Scheduling of vehicles for a central depot to a number of delivery points, Oper. Res. 12 (1964) 568–581.

[5] M.L. Fisher, Optimal solution of vehicle routing problems using minimum k-trees, Oper. Res. 42 (4) (1994) 626–646.
[6] B. Fleischmann, The vehicle routing problem with multiple use of vehicles, Working paper, Fachbereich Wirschaftswissenschaften, Universitat Hamburg, 1990.
[7] B. Golden, G. Laporte, E. Taillard, An adaptive memory heuristic for a class of vehicle routing problems with minmax objective, Comput. Oper. Res. 24 (1997) 445–452.
[8] S. Martello, P. Toth, Knapsack Problems, Wiley, Chichester, 1990.
[9] V.J. Rayward-Smith, I.H. Osman, C.R. Reeves, G.D. Smith (Eds.), Local search strategies for the mix fleet routing problem, in: Modern Heuristic Search Methods, Wiley, New York, 1996.
[10] Y. Rochat, E. Taillard, Probabilistic diversification and intensification in local search for vehicle routing, Heuristics 1 (1995) 147–167.
[11] S. Salhi, The integration of routing into the location-allocation and vehicle composition problems, Ph.D. Thesis, University of Lancaster, 1987, pp. 198–208.
[12] S. Salhi, G.K. Rand, Improvements to vehicle routing heursitics, J. Oper. Res. Soc. 38 (1987) 293–295.
[13] S. Salhi, M. Sari, A multi-level composite heuristic for the multi-depot vehicle fleet mix problem, Eur. J. Oper. Res. 103 (1997) 95–112.
[14] E. Taillard, G. Laporte, M. Gendreau, Vehicle routing with multiple use of vehicles, J. Oper. Res. Soc. 47 (1996) 1065–1070.
[15] P.C. Yellow, A computational modification to the savings method of vehicle scheduling, Oper. Res. Q. 21 (1970) 281–283.